File Edit Format Run Options Window Help

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.preprocessing import OneHotEncoder

try:
    df = pd.read_csv("agriculture_data.csv")
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("File not found. Generating a mock dataset for demonstration...")

    np.random.seed(42)
    mock_data = {
        "Crop": np.random.choice(["Rice", "Wheat", "Maize", "Cotton"], 1000),
        "Variety": np.random.choice(["Hybrid", "Local", "Fine"], 1000),
        "State": np.random.choice(
            ["Punjab", "Uttar Pradesh", "Andhra Pradesh", "Tamil Nadu"], 1000
        ),
        "Quantity": np.random.randint(50, 500, 1000),
        "production": np.random.randint(
            100, 2000, 1000
        ),  # Assuming target variable
        "Season": np.random.choice(["Medium", "Long"], 1000),
        "Unit": ["Tons"] * 1000,
        "Cost": np.random.randint(5000, 30000, 1000),
        "Recommended Zone": np.random.choice(
            ["Zone A", "Zone B", "Zone C"], 1000
        ),
    }
    df = pd.DataFrame(mock_data)


print("\n--- Initial Data Info ---")
print(df.info())


if "Unit" in df.columns:
    df = df.drop(columns=["Unit"])

# Handle missing values if any exist
# Separate Features (X) and Target (y). Assuming we are predicting 'production'
```

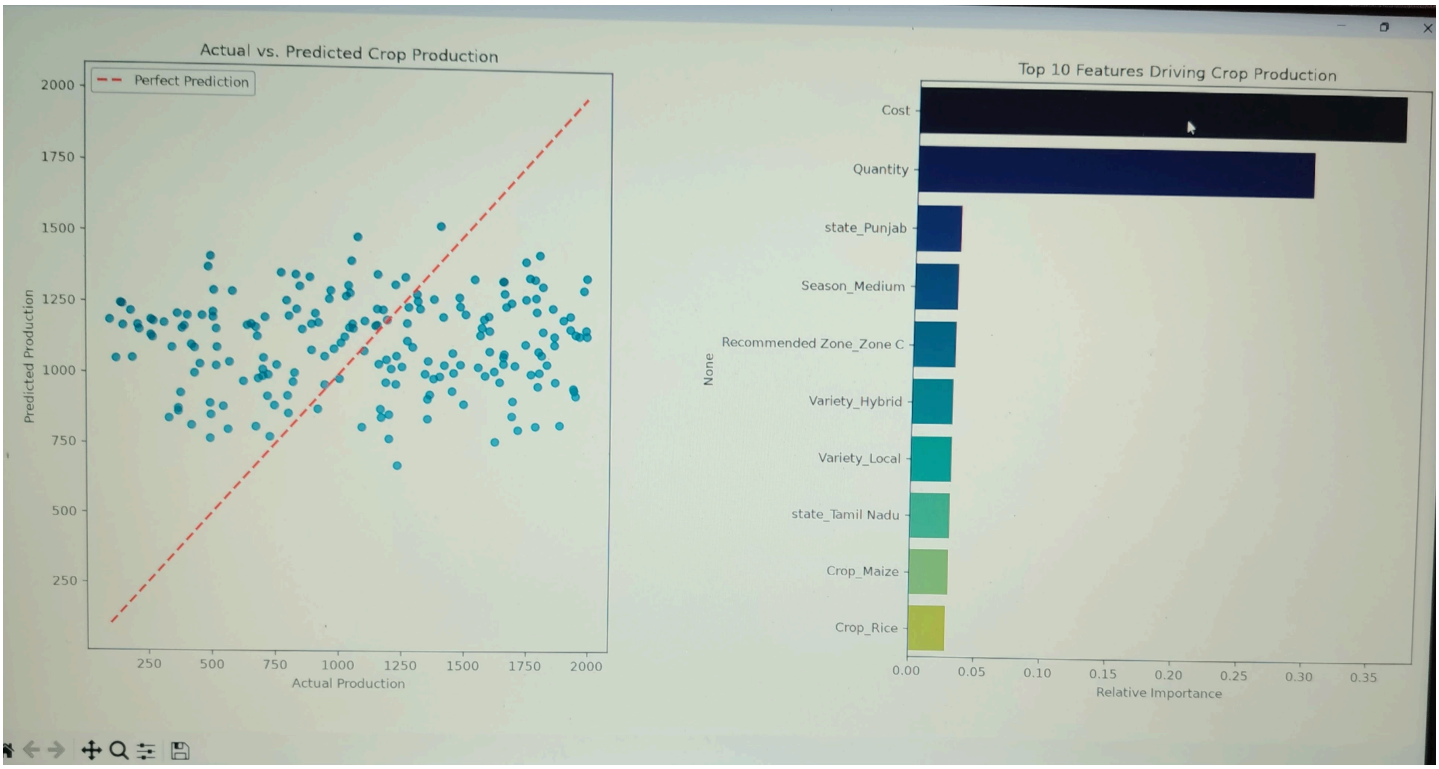
File Edit Format Run Options Window Help

```
y_pred = rf_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n--- Model Performance Metrics ---")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R2) Score: {r2:.4f} ({r2*100:.2f}%)")


plt.figure(figsize=(12, 5))

# Plot 1: Actual vs Predicted
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.6, color="teal")
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    "r--",
    lw=2,
    label="Perfect Prediction",
)
plt.xlabel("Actual Production")
plt.ylabel("Predicted Production")
plt.title("Actual vs. Predicted Crop Production")
plt.legend()

# Plot 2: Top 10 Feature Importances
plt.subplot(1, 2, 2)
importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1][:10]  # Top 10 features
sns.barplot(
    x=importances[indices],
    y=X_final.columns[indices],
    hue=X_final.columns[indices],
    palette="viridis",
    legend=False,
)
plt.xlabel("Relative Importance")
plt.title("Top 10 Features Driving Crop Production")

plt.tight_layout()
plt.show()
```

Ln: 1 Col: 0

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\gshak\agriculture.py ==========
File not found. Generating a mock dataset for demonstration...

--- Initial Data Info ---
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Crop             1000 non-null   str
 1   Variety          1000 non-null   str
 2   state            1000 non-null   str
 3   Quantity         1000 non-null   int32
 4   production       1000 non-null   int32
 5   Season           1000 non-null   str
 6   Unit             1000 non-null   str
 7   Cost             1000 non-null   int32
 8   Recommended Zone 1000 non-null   str
dtypes: int32(3), str(6)
memory usage: 58.7 KB
None

Warning (from warnings module):
  File "C:\Users\gshak\agriculture.py", line 52
    cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future ver
sion. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

Numerical Features: []
Categorical Features: ['Crop', 'Variety', 'state', 'Season', 'Recommended Zone']

Training set size: 800 samples
Testing set size: 200 samples

Training the Random Forest Regressor model...

--- Model Performance Metrics ---
Mean Absolute Error (MAE): 464.04
Root Mean Squared Error (RMSE): 542.68
R-squared (R2) Score: -0.0455 (-4.55%)
```


Actual vs. Predicted Crop Production
Perfect Prediction
Predicted Production
Actual Production
2000
1750
1500
1250
1000
750
500
250
Top 10 Features Driving Crop Production
Cost
Quantity
state_Punjab
Season_Medium
Recommended Zone_Zone C
Variety_Hybrid
Variety_Local
state_Tamil Nadu
Crop_Maize
Crop_Rice
None
0.00
0.05
0.10
0.15
0.20
0.25
0.30
0.35
Relative Importance

File Edit Format Run Options Window Help

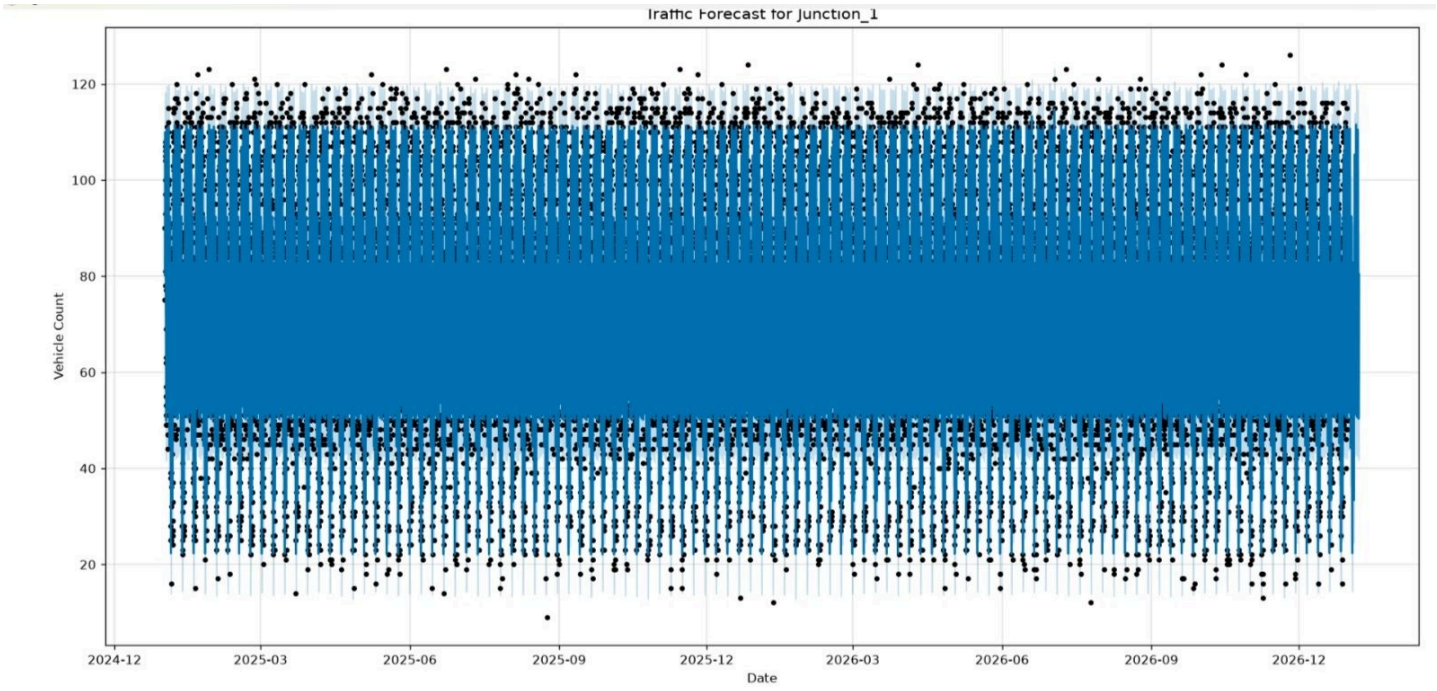
```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.ensemble import RandomForestRegressor
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.preprocessing import OneHotEncoder

try:
    df = pd.read_csv("agriculture_data.csv")
    print("Dataset loaded successfully!")
except FileNotFoundError:
    print("File not found. Generating a mock dataset for demonstration...")

    np.random.seed(42)
    mock_data = {
        "Crop": np.random.choice(["Rice", "Wheat", "Maize", "Cotton"], 1000),
        "Variety": np.random.choice(["Hybrid", "Local", "Fine"], 1000),
        "state": np.random.choice(
            ["Punjab", "Uttar Pradesh", "Andhra Pradesh", "Tamil Nadu"], 1000
        ),
        "Quantity": np.random.randint(50, 500, 1000),
        "production": np.random.randint(
            100, 2000, 1000
        ),  # Assuming target variable
        "Season": np.random.choice(["Medium", "Long"], 1000),
        "Unit": ["Tons"] * 1000,
        "Cost": np.random.randint(5000, 30000, 1000),
        "Recommended Zone": np.random.choice(
            ["Zone A", "Zone B", "Zone C"], 1000
        ),
    }
    df = pd.DataFrame(mock_data)


print("\n--- Initial Data Info ---")
print(df.info())


if "Unit" in df.columns:
    df = df.drop(columns=["Unit"])

# Handle missing values if any exist
# Separate Features (X) and Target (y). Assuming we are predicting 'production'
X = df.drop(columns=["production"])
```

File Edit Format Run Options Window Help

```
y_pred = rf_model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("\n--- Model Performance Metrics ---")
print(f"Mean Absolute Error (MAE): {mae:.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.2f}")
print(f"R-squared (R2) Score: {r2:.4f} ({r2*100:.2f}%)")


plt.figure(figsize=(12, 5))

# Plot 1: Actual vs Predicted
plt.subplot(1, 2, 1)
plt.scatter(y_test, y_pred, alpha=0.6, color="teal")
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    "r--",
    lw=2,
    label="Perfect Prediction",
)
plt.xlabel("Actual Production")
plt.ylabel("Predicted Production")
plt.title("Actual vs. Predicted Crop Production")
plt.legend()

# Plot 2: Top 10 Feature Importances
plt.subplot(1, 2, 2)
importances = rf_model.feature_importances_
indices = np.argsort(importances)[::-1][:10]  # Top 10 features
sns.barplot(
    x=importances[indices],
    y=X_final.columns[indices],
    hue=X_final.columns[indices],
    palette="viridis",
    legend=False,
)
plt.xlabel("Relative Importance")
plt.title("Top 10 Features Driving Crop Production")

plt.tight_layout()
plt.show()
```

File Edit Shell Debug Options Window Help

```
Python 3.13.14 (tags/v3.13.14:fd17997, Jun 10 2026, 13:03:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:\Users\gshak\agriculture.py ==========
File not found. Generating a mock dataset for demonstration...

--- Initial Data Info ---
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Crop              1000 non-null   str
 1   Variety           1000 non-null   str
 2   state             1000 non-null   str
 3   Quantity          1000 non-null   int32
 4   production        1000 non-null   int32
 5   Season            1000 non-null   str
 6   Unit              1000 non-null   str
 7   Cost              1000 non-null   int32
 8   Recommended Zone  1000 non-null   str
dtypes: int32(3), str(6)
memory usage: 58.7 KB
None

Warning (from warnings module):
  File "C:\Users\gshak\agriculture.py", line 52
    cat_cols = X.select_dtypes(include=["object"]).columns.tolist()
Pandas4Warning: For backward compatibility, 'str' dtypes are included by select_dtypes when 'object' dtype is specified. This behavior is deprecated and will be removed in a future ver
sion. Explicitly pass 'str' to `include` to select them, or to `exclude` to remove them and silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-migration-select-dtypes for details on how to write code that works with pandas 2 and 3.

Numerical Features: []
Categorical Features: ['Crop', 'Variety', 'state', 'Season', 'Recommended Zone']

Training set size: 800 samples
Testing set size: 200 samples

Training the Random Forest Regressor model...

--- Model Performance Metrics ---
Mean Absolute Error (MAE): 464.04
Root Mean Squared Error (RMSE): 542.68
R-squared (R2) Score: -0.0455 (-4.55%)
```


Traffic Forecast for Junction_1
Vehicle Count
120
100
80
60
40
20
2024-12
2025-03
2025-06
2025-09
2025-12
2026-03
2026-06
2026-09
2026-12
Date